Министерство образования и науки Российской Федерации
Уральский федеральный
университет имени первого
Президента России Б. Н. Ельцина

Программная инженерия

Отчет

Практическая работа
«Мониторинг доступности API, анализ логов
и контейнеризация сервиса»

Выполнил студент
Гр. РИМ-160980к
Синцев Н. А.

Преподаватель
Меркушов М.

Екатеринбург, 2026

1. Git Workflow и организация проекта

Создан публичный репозиторий infra-monitoring-project на GitHub. Ветка main защищена правилом: изменения в неё вносятся только через Pull Request. Вся работа велась в ветке feature/monitoring, изменения оформлены отдельными коммитами по этапам.

Ссылка на репозиторий: https://github.com/SincN/infra-monitoring-project

Структура проекта:

– scripts/check_api.sh — скрипт проверки доступности API;

– scripts/analyze_logs.sh — скрипт генерации и анализа access.log;

– docker-compose.yml, nginx.conf — описание стека мониторинга;

– .github/workflows/ci.yml — CI/CD пайплайн;

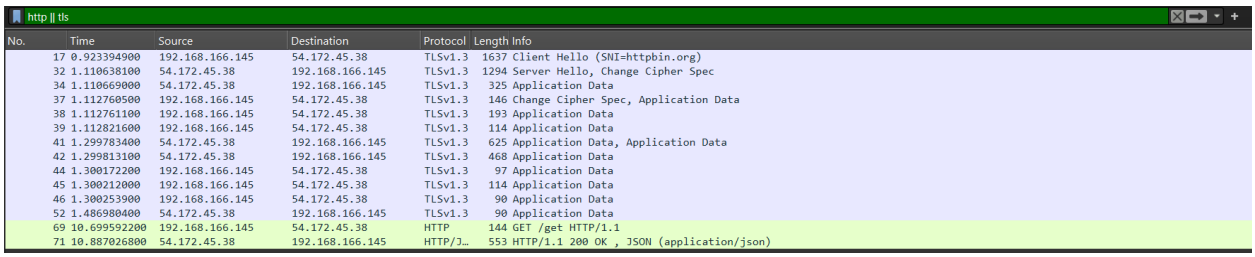
– README.md — описание проекта, инструкции по запуску и деплою через SSH/SCP;

– .gitignore — исключает логи (*.log) и сгенерированный access.log.

По завершении работы создан Pull Request #1 «Мониторинг доступности echo-сервера» из feature/monitoring в main с описанием изменений.

2. Анализ сетевого трафика

Захват трафика выполнялся в Wireshark на интерфейсе vEthernet (WSL), так как запросы отправлялись из Ubuntu (WSL) командой curl к публичному API httpbin.org — по протоколу HTTP (http://httpbin.org/get) и HTTPS (https://httpbin.org/get). Для отображения применён фильтр http || tls. Дамп сохранён в файл dump.pcapng и приложен к отчёту.


http || tls

No.	Time	Source	Destination	Protocol	Length	Info
17	0.923394900	192.168.166.145	54.172.45.38	TLSv1.3	1637	Client Hello (SNI=httpbin.org)
32	1.110638100	54.172.45.38	192.168.166.145	TLSv1.3	1294	Server Hello, Change Cipher Spec
34	1.110669000	54.172.45.38	192.168.166.145	TLSv1.3	325	Application Data
37	1.112760500	192.168.166.145	54.172.45.38	TLSv1.3	146	Change Cipher Spec, Application Data
38	1.112761100	192.168.166.145	54.172.45.38	TLSv1.3	193	Application Data
39	1.112821600	192.168.166.145	54.172.45.38	TLSv1.3	114	Application Data
41	1.299783400	54.172.45.38	192.168.166.145	TLSv1.3	625	Application Data, Application Data
42	1.299813100	54.172.45.38	192.168.166.145	TLSv1.3	468	Application Data
44	1.300172200	192.168.166.145	54.172.45.38	TLSv1.3	97	Application Data
45	1.300212000	192.168.166.145	54.172.45.38	TLSv1.3	114	Application Data
46	1.300253900	192.168.166.145	54.172.45.38	TLSv1.3	90	Application Data
52	1.486980400	54.172.45.38	192.168.166.145	TLSv1.3	90	Application Data
69	10.699592200	192.168.166.145	54.172.45.38	HTTP	144	GET /get HTTP/1.1
71	10.887026800	54.172.45.38	192.168.166.145	HTTP/J…	553	HTTP/1.1 200 OK , JSON (application/json)

Рисунок 1 — Отфильтрованный HTTP/HTTPS-трафик в Wireshark

IP-адреса, участвовавшие в обмене:

– 192.168.166.145 — клиент (Ubuntu в WSL), с него отправлялись

запросы;

– 192.168.160.1 — шлюз WSL, выполнял роль DNS-сервера: на запрос имени httpbin.org вернул адреса 54.172.45.38, 44.209.237.184, 3.221.253.123;
– 54.172.45.38 — сервер httpbin.org, с которым установлены соединения на порт 80 (HTTP) и порт 443 (HTTPS).

Использованные HTTP-методы:

– GET — запрос GET /get HTTP/1.1 (Host: httpbin.org).

Коды ответов сервера:

– 200 OK — успешный ответ, тело в формате JSON (application/json).

HTTPS-соединение защищено протоколом TLS 1.3. В дампе видны только IP-адреса, порт 443, TLS-рукопожатие (Client Hello, Server Hello) и имя сервера в открытом виде (SNI = httpbin.org). Метод, URL и код ответа передаются в зашифрованных пакетах Application Data и в Wireshark не читаются. В отличие от этого, HTTP-запрос по порту 80 передаётся открытым текстом, и его метод и код ответа видны полностью.

3. Bash-скрипты для мониторинга и анализа

3.1. Скрипт мониторинга API (check_api.sh)

Скрипт принимает URL как аргумент командной строки, проверяет количество аргументов и формат URL (регулярное выражение ^https?://), выполняет запрос через curl и получает HTTP-код ответа (curl -s -o /dev/null -w '%{http_code}'). При коде 200 выводит «API доступен» и завершается с кодом возврата 0, иначе — «API недоступен, код ответа: XXX» и код возврата 1.

Проверка:

– https://httpbin.org/get — «API доступен (код ответа: 200)», код возврата 0;
– https://blablabla.org/get — «API недоступен, код ответа: 000», код возврата 1. Код 000 означает, что HTTP-ответ не получен: соединение с хостом установить не удалось (curl завершился с ошибкой 35 — ошибка установления SSL/TLS-соединения).

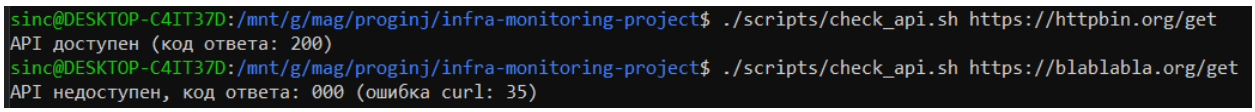


Рисунок 2 — Работа check_api.sh с валидным и невалидным URL

3.2. Скрипт анализа логов (analyze_logs.sh)

Скрипт генерирует тестовый файл access.log на 600 строк. IP-адрес, HTTP-метод, URL и код ответа для каждой записи выбираются случайно из заранее заданных массивов. Пример записи:

```
192.168.1.10 - - [15/Aug/2026:10:00:00] "GET /api/auth/users HTTP/1.1"
200
```

Затем выполняется анализ:

– общее количество запросов — wc -l;

– топ-3 IP-адреса — IP извлекаются командой grep -oE по регулярному выражению ^[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}, затем конвейер sort | uniq -c | sort -nr | head -3 подсчитывает повторы и выбирает три самых частых;

– количество запросов с кодами 4xx и 5xx — awk проверяет последнее поле строки ($NF) регулярными выражениями ^4[0-9][0-9]$ и ^5[0-9][0-9]$, результат считает wc -l.

Результат: всего 600 запросов; топ-3 IP — 172.16.0.3 (132), 10.0.0.7 (127), 192.168.1.10 (120); запросов с кодами 4xx — 211, с кодами 5xx — 139.

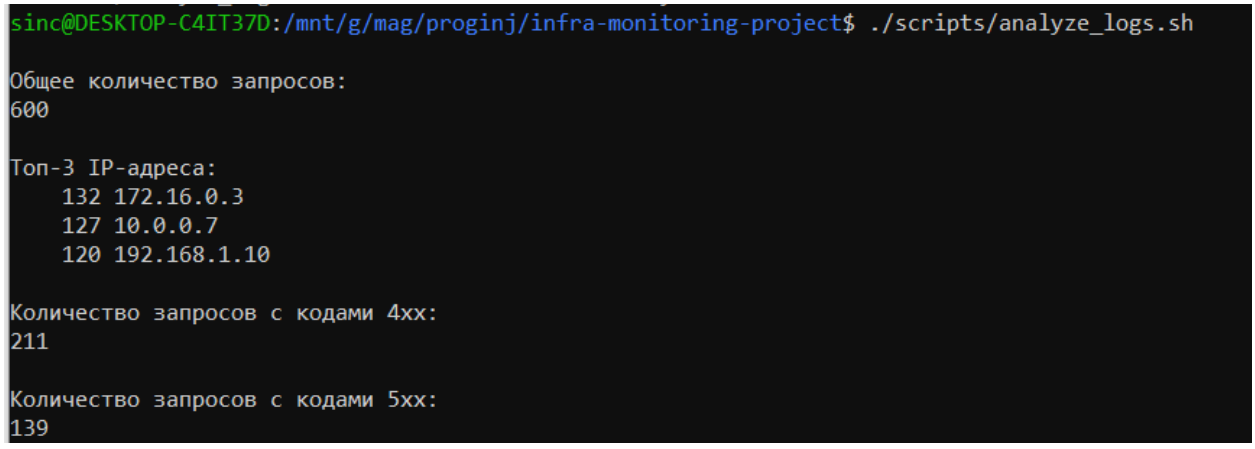


Рисунок 3 — Вывод работы analyze_logs.sh

Оба скрипта сделаны исполняемыми (chmod +x) и протестированы.

4. Docker Compose для стека мониторинга

В файле docker-compose.yml описаны два сервиса на основе готовых

образов из Docker Hub:

– api — образ hashicorp/http-echo, запускается с параметрами -listen=:8080 -text="Monitoring Service Active" и отвечает этим текстом на порту 8080;

– proxy — образ nginx:alpine, публикует порт 80 на хост (80:80) и проксирует запросы на сервис api. Конфигурация nginx (nginx.conf) подключается в контейнер через volume; в ней директива proxy_pass http://api:8080 пересылает все запросы на echo-сервис.

Для связи контейнеров создана внутренняя сеть monitoring-net (драйвер bridge). Внутри сети nginx обращается к echo-сервису по имени сервиса api — его разрешает встроенный DNS Docker. Порт 8080 сервиса api на хост не публикуется, поэтому единственная точка входа — прокси.

Стек запущен командой docker compose up -d. Команда docker compose ps показывает оба контейнера в состоянии Up, запрос curl http://localhost:80 возвращает «Monitoring Service Active» — то есть запрос прошёл через nginx до echo-сервиса.

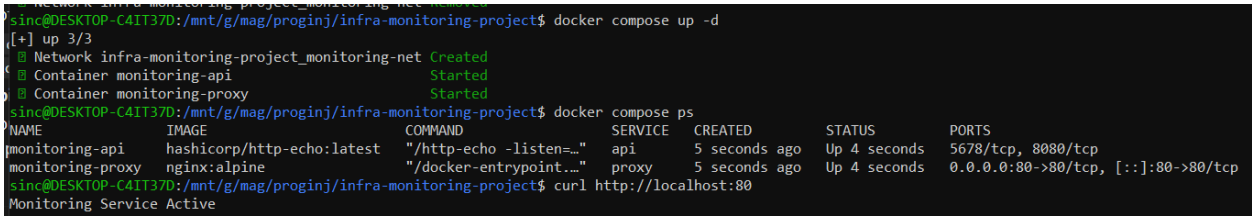


Рисунок 4 — Работающие контейнеры (docker compose ps) и ответ сервиса

5. CI/CD Pipeline (GitHub Actions)

Пайплайн описан в файле .github/workflows/ci.yml и запускается при создании Pull Request в ветку main. Задача выполняется на виртуальной машине ubuntu-latest:

– Checkout — загрузка кода репозитория;

– Chmod — выдача скриптам прав на выполнение;

– Syntax — проверка синтаксиса: bash -n check_api.sh и bash -n analyze_logs.sh;

– Test check_api.sh — запуск check_api.sh https://httpbin.org/status/200. Шаг успешен только при коде возврата 0, при любом другом коде пайплайн падает.

Пайплайн для Pull Request #1 прошёл успешно.

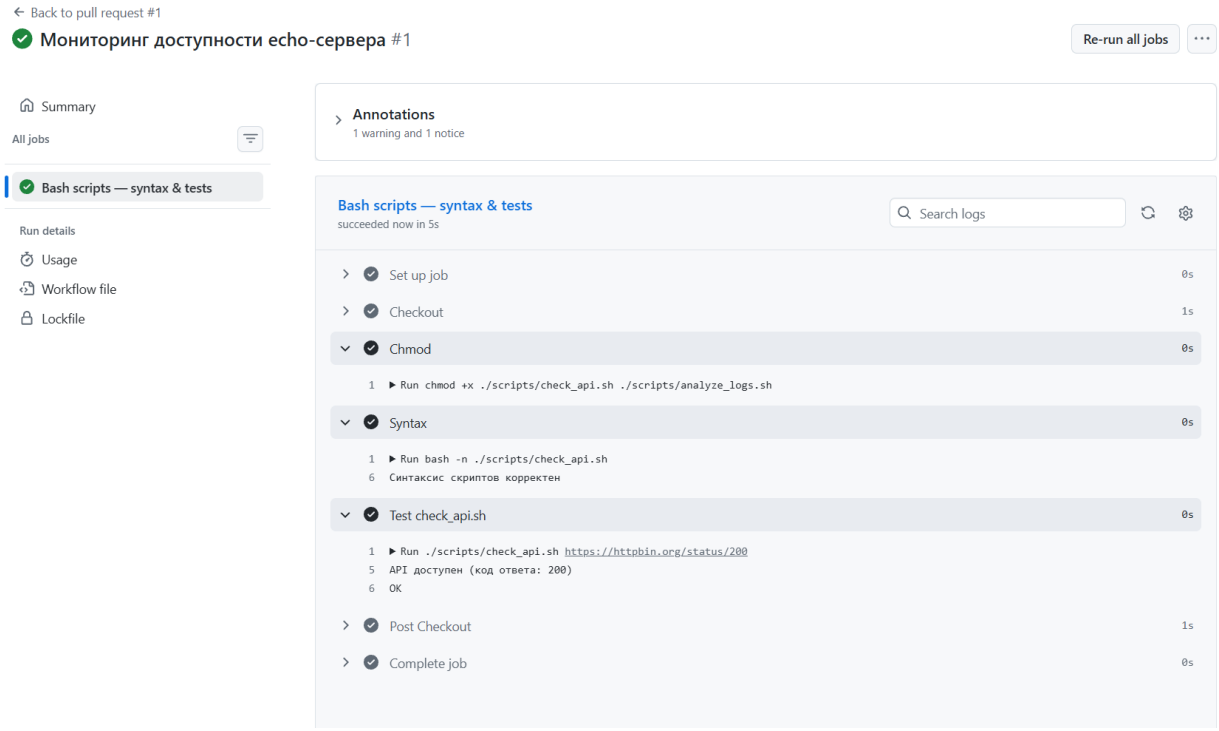


Рисунок 5 — Успешное выполнение CI/CD пайплайна в GitHub Actions

6. SSH и SCP

В README.md описан процесс безопасной передачи проекта на удалённый сервер:

– генерация пары ключей: ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa (приватный ключ остаётся у пользователя, публичный передаётся на сервер);

– копирование публичного ключа на сервер: ssh-copy-id -i ~/.ssh/id_rsa.pub user@remote_host (ключ добавляется в ~/.ssh/authorized_keys, после чего вход выполняется без пароля);

– передача проекта командой scp -r ./infra-monitoring-project user@remote_host:~/ или архивом через tar + ssh;

– запуск на сервере: ssh user@remote_host, затем docker compose up -d.

```
tar -czf - --exclude=.git infra-monitoring-project | ssh
user@remote_host "tar -xzf - -C ~/"
```

Вывод

В ходе работы создан репозиторий с Git workflow через рабочую ветку

и Pull Request в защищённую ветку main. С помощью Wireshark проанализирован трафик к публичному API: у HTTP-запросов видны методы и коды ответов, а у HTTPS-трафика, зашифрованного TLS, видны только адреса, рукопожатие и имя сервера. Написаны Bash-скрипты для проверки доступности API и анализа логов с использованием конвейеров, grep, awk, sort, uniq и регулярных выражений. Развёрнут стек из echo-сервиса и nginx-прокси в Docker Compose с общей внутренней сетью. Настроен CI/CD пайплайн GitHub Actions, который автоматически проверяет синтаксис скриптов и работу скрипта мониторинга при каждом Pull Request. Описан процесс безопасного деплоя проекта на сервер по SSH/SCP.